

StoreSense-AI · AI Representation Optimizer

Product Document · StarX · Kasparro Hackathon 2026

1. Problem Statement

AI shopping assistants — ChatGPT Search, Perplexity, Google Gemini — are replacing traditional search as a product-discovery surface. They don't rank keywords; they reason about stores. A store that reads perfectly to a human may be opaque or untrustworthy to an LLM agent, and the merchant will never know — silently losing high-intent traffic with no attribution.

The Gap: No tool exists that simulates, scores, or explains AI recommendation confidence for Shopify merchants.

- AI shopping traffic is growing faster than traditional search. The revenue loss is silent and unattributed.
- The gap is structural — thin policies, missing semantics, inconsistent pricing are invisible to SEO tools but decisive for LLMs.
- No feedback loop exists between store data quality and AI recommendation outcomes.

2. Target User

Who: Shopify store owners and e-commerce managers who track channel performance and can act on structured recommendations.

- Currently rely on SEO tools (Ahrefs, SEMrush) — optimised for human keywords, completely blind to LLM reasoning.
- When AI traffic underperforms there is no diagnostic surface. The failure mode is invisible.
- Recommendations, when they exist, come from consultants — anecdotal, unscored, unprioritised.

The three questions they cannot answer today:

“Why is my store not appearing in AI product summaries?”

“Is my product data professional enough for an AI agent to recommend me?”

“Which fixes will actually move the needle — and in what order?”

3. What We Built & Core User Journey

StoreSense-AI is a **stateless diagnostic engine** that audits a Shopify store through three lenses — Data Completeness, Trust Signals, and Semantic Perception — returning a scored, prioritised optimisation roadmap.

Step	Stage	What happens	Output
01	Input	Merchant provides Shopify store URL.	—

02	Ingest	Top 50 active products + metadata pulled via Shopify Admin API. Capped at 50 for sub-15s response.	Raw data
03	Analyse	(a) Completeness — missing descriptions, images, tags; (b) Trust — presence & depth of Return, Shipping, Privacy policies; (c) Perception — LLM as AI shopping assistant surfaces buyer objections.	3 signals
04	Score	Weighted engine: Completeness 35% + Trust 35% + Perception 30% = AI Readiness Score (0–100).	Score
05	Output	Scorecard · Perception Brief (confidence level + named objections) · Prioritised Roadmap (High/Med/Quick Wins) · What-If projection.	Dashboard

4. Key Product Decisions

Decision	Reasoning	Code Evidence
Hybrid scoring: rules + LLM	Rules catch binary gaps (missing image, no description). LLMs catch semantic issues no rule set can encode.	scoring_rules.py splits weights: completeness (deterministic) vs. perception (LLM).
What-If score projection	Merchants need motivation to act, not just a diagnosis. Showing potential score makes cost of inaction concrete.	_calculate_what_if in recommendation.py simulates a perfect-state store.
Policy depth, not just presence	A 50-word policy is useless to an LLM. AI agents need substantive language to infer trustworthiness.	MIN_POLICY_THRESHOLD = 300 in trust.py classifies Weak vs. Present.
Pro-merchant error handling	API timeout = our failure, not the merchant's. They shouldn't be penalised for our infrastructure.	Unknown status → zero score penalty in scoring_rules.py.
Quick Wins as a priority tier	Long fix lists cause disengagement. Sub-30-min wins drive the early engagement loop.	RECOMMENDATION_PROMPT enforces a Quick Wins section in every output.

5. What We Chose Not to Build

Every exclusion below is a deliberate decision made under time and complexity constraints — not an omission.

Excluded	Reasoning
Persistent database / history	Storage introduces GDPR surface area for merchant inventory data. Stateless = zero data liability. Correct for a diagnostic MVP.
Full-catalogue crawl (>50 products)	500–5,000 products pushes latency beyond HTTP timeout, forcing async architecture. 50 products deliver representative signal with synchronous UX.
Auto-fix / write-back to Shopify	Modifying live inventory without approval is high-risk. Read-only contract keeps merchants in control. Safety first.
Real-time monitoring / re-scoring	Requires webhooks, scheduler, and delta detection. High complexity, low MVP value. One great diagnostic pass is the right first product.
Multi-page site crawling	Blog posts and CMS content add complexity and latency. Admin API product data = highest signal, smallest surface area.

6. Tradeoffs

Tension	Choice Made	How We Resolved It
Accuracy vs. Latency	50 products, not full catalogue	50 products give representative coverage. Output labelled 'sampled analysis' so scope is transparent to the merchant.
Cost vs. Differentiation	Keep LLM perception layer	Rules-only scoring is cheap but commoditised. LLM perception captures semantic trust — our core moat. Worth the per-request cost.
Simulation vs. Real Behaviour	Simulate AI agent reasoning	Real AI traffic attribution doesn't exist yet. We use Gemini — same model family as agents we optimise for — giving proxy validity.
Depth vs. Speed (policy scoring)	Word-count threshold as proxy	300+ word policies almost always contain substantive terms. Fast, defensible proxy without needing a dedicated classifier at MVP.

Core Insight: Traditional SEO optimises for visibility. AI commerce optimises for trust and recommendation confidence. A store is ranked by whether an AI agent is confident enough to stake its recommendation on it. StoreSense-AI measures and improves that confidence directly.